

Строковый тип данных (String, str) в Python

Что такое строка?

Строка (str) — это упорядоченная последовательность символов (букв, цифр, знаков препинания, пробелов и т.д.), заключенная в кавычки. В Python строки используются для хранения и обработки текстовой информации.

Основные характеристики

1. Упорядоченность (Sequence)

Символы в строке имеют определенный порядок и могут быть доступны по **индексу**, начиная с **0**.

- **Индексация:**
 - Прямая: от **0** (первый символ) до **N-1** (последний символ).
 - Обратная: от **-1** (последний символ) до **-N**.
- **Срезы (Slicing):** Позволяют извлекать подстроки по заданным начальному и конечному индексам (например, `строка[начало:конец:шаг]`).

2. Неизменяемость (Immutable)

Строки в Python являются **неизменяемым** типом данных. Это означает, что после создания строки вы **не можете** изменить, добавить или удалить отдельный символ внутри нее.

🛑 Если вы "изменяете" строку, Python на самом деле создает совершенно **новую** строку в памяти, а старая остается нетронутой (или будет удалена сборщиком мусора).

3. Назначение

Строки необходимы для:

- Хранения имен, описаний, сообщений, URL.
- Работы с текстовыми файлами (чтение, запись).
- Форматирования вывода для пользователя.

Какие кавычки использовать?

Python предоставляет несколько вариантов, чтобы избежать конфликтов при использовании кавычек внутри строки:

Тип кавычек	Пример	Когда использовать
Одинарные	'Привет'	Самый распространенный, если внутри нет апострофов.

Двойные	"Hello"	Удобно, если внутри есть апострофы/одинарные кавычки (например, "I'm happy").
Тройные	"""Много строк"""	Используются для создания многострочных строк и для написания документации (docstrings).

Основные методы и операции работы со строками

Категория	Операция/Метод	Описание	Пример
Конкатенация	+	Объединяет две строки.	'Py' + 'thon' 'Python'
Дублирование	*	Повторяет строку N раз.	'Aga' * 3 'AgaAgaAga'
Длина	len()	Встроенная функция, возвращает количество символов.	len('код') 3
Регистр	.upper()	Переводит все символы в верхний регистр.	"привет".upper() → "ПРИВЕТ"
	.lower()	Переводит все символы в нижний регистр.	"ПРИВЕТ".lower() → "привет"
	.capitalize()	Делает первую букву заглавной, остальные — строчными.	"python".capitalize() → "Python"
	.title()	Делает заглавной первую букву каждого слова.	"привет мир".title() → "Привет Мир"

	<code>.swapcase()</code>	Меняет регистр на противоположный (заглавные в строчные и наоборот).	<code>"PyThOn".swapcase()</code> → <code>"pYtHoN"</code>
Проверка	<code>.isalpha()</code>	True, если строка состоит только из букв.	<code>"Hello".isalpha()</code> → True
	<code>.isdigit()</code>	True, если строка состоит только из цифр.	<code>"12345".isdigit()</code> → True
	<code>.isalnum()</code>	True, если строка состоит из букв и/или цифр.	<code>"id123".isalnum()</code> → True
	<code>.startswith(s)</code>	True, если строка начинается с подстроки s.	<code>"Python".startswith("Py")</code> → True
	<code>.endswith(s)</code>	True, если строка заканчивается подстрокой s.	<code>"image.png".endswith(".png")</code> → True
Поиск	<code>.find(s)</code>	Возвращает индекс первого вхождения s (или -1, если не найдено).	<code>"banana".find("a")</code> → 1
	<code>.count(s)</code>	Считает, сколько раз подстрока s встречается в строке.	<code>"banana".count("a")</code> → 3

	<code>.replace(old, new)</code>	Заменяет все вхождения <code>old</code> на <code>new</code> .	<code>"apple".replace("p", "b") → "abble"</code>
Разделение	<code>.split(d)</code>	Разрезает строку на список по разделителю <code>d</code> (по умолчанию — пробел).	<code>"1,2,3".split(",") → ['1', '2', '3']</code>
	<code>.join(list)</code>	Соединяет элементы списка в строку, используя текущую строку как разделитель.	<code>"-".join(["a", "b"]) → "a-b"</code>
Очистка	<code>.strip()</code>	Удаляет пробельные символы (или указанные знаки) с обоих концов.	<code>" hi ".strip() → "hi"</code>
	<code>.lstrip()</code> <code>.rstrip()</code>	Удаляет символы только слева или только справа.	<code>"!!wow".lstrip("!") → "wow"</code>
Формат	<code>.zfill(width)</code>	Добавляет нули слева, пока строка не достигнет длины <code>width</code> .	<code>"42".zfill(5) → "00042"</code>

Примеры работы со строками

Пример 1: Индексация и срезы

```
message = "Python"

# Индексация
first_char = message[0] # 'P'
last_char = message[-1] # 'n'
```

```
# Срез (элементы с индекса 2 по 4, исключая 5-й)
sub_string = message[2:5] # 'tho'

# Срез с шагом (разворот строки)
reversed_string = message[::-1] # 'nohtyP'
```

Пример 2: Использование методов

```
# 1. Замена и регистр
email = "USER@Example.com"
email_lower = email.lower() # 'user@example.com'

# 2. Разделение и соединение
tags_str = "программирование,python,урок"
tags_list = tags_str.split(',') # ['программирование',
'python', 'урок']
new_tags_str = " | ".join(tags_list) # 'программирование
| python | урок'

# 3. Проверка содержимого
is_numeric = "123".isdigit() # True
is_alpha = "Hello".isalpha() # True
```

Практические задачи для занятия

Легкий уровень (5-10 минут)

1. Сложение строк:

Создайте две переменные: `name = "Алексей"` и `greeting = "Привет, "`. Объедините их с помощью оператора `+` и сохраните результат в новой переменной `full_message`. Выведите `full_message`.

2. Проверка длины:

Дана строка `word = "абстракция"`. Используя функцию `len()`, выведите, сколько символов в этой строке.

3. Перевод в верхний регистр:

Дана строка `secret = "кодовое слово"`. Используя метод `upper()`, преобразуйте ее в верхний регистр и выведите результат.

Средний уровень (10-15 минут)

1. Обработка данных пользователя:

Попросите пользователя ввести его имя и фамилию через пробел (например, "Иван Петров"). Разделите введенную строку на два слова, используя метод `split()`, и выведите имя и фамилию отдельно, каждое с большой буквы (используйте `capitalize()`).

2. Извлечение URL:

Дана строка `full_url = "https://www.example.com/page/home"`.

Используя срезы, выведите только доменное имя: `"www.example.com"`.

3. Замена символов:

Дана строка `phone = "8-800-555-35-35"`. Замените все дефисы (-) на пробелы (), используя метод `replace()`, и сохраните результат.

Контрольные вопросы

1. Что такое строка в Python?
2. Является ли строковый тип данных изменяемым (mutable) или неизменяемым (immutable)? Что это значит на практике?
3. Для чего используется тройные кавычки ("""...""")?
4. Какой индекс имеет первый символ в строке? Какой индекс имеет последний символ при использовании обратной индексации?
5. Если у нас есть список слов `['apple', 'banana', 'cherry']`, какой метод строк нужно использовать, чтобы объединить их в одну строку через запятую и пробел (,)?
6. Как узнать количество символов в строке?
7. Что вернет метод `find()`, если искомая подстрока не найдена?